

1. Entendimento do Negócio

Nesta fase inicial, definimos os objetivos estratégicos do projeto. A premissa fundamental é a transição de um modelo de dados fragmentado para uma arquitetura de dados unificada e auditada.

1.1. Objetivos de Negócio

O foco principal é **criar uma fonte única de faturamento (Single Source of Truth)** para o desenvolvimento de campanhas de vendas e análises gerenciais.

- **Genérica e Escalável**, permitindo que a organização supere os seguintes desafios:
- **Fragmentação Transacional**: Consolidar um histórico vasto que hoje se encontra disperso em múltiplos arquivos e fontes.
- **Identificação de Potencial**: Facilitar a localização de clientes com alto potencial de compra através da unificação de suas diferentes identidades (CNPJs).
- **Monitoramento de Market Share**: Permitir uma visão real da participação da INOVA dentro de cada cliente/grupo, combatendo a visão míope causada por cadastros duplicados.

1.2. Avaliação da Situação Atual

O cenário base apresenta extrações brutas do Power BI que, apesar de ricas em volume, possuem baixa qualidade de saneamento:

1. **Inconsistência de Identidade**: O mesmo cliente é tratado como entidades diferentes devido a variações cadastrais e filiais.
2. **Complexidade Financeira**: Dificuldade em conciliar automaticamente vendas e devoluções para obter o faturamento líquido real.
3. **Dependência de Processos Lentos**: Processamento manual de arquivos Excel que gera gargalos operacionais.

1.3. Objetivos de Dados (Visão Técnica)

Para sustentar o objetivo de negócio, o pipeline entrega:

- **Motor de Identidade Global**: Algoritmo agnóstico que agrupa empresas por DNA nominal, independente da campanha.
- **Governança de Dados**: Auditoria de continuidade para garantir que não existam lacunas ("gaps") no histórico transacional.
- **Alta Performance**: Entrega de um "Dataset de Ouro" em formato `.parquet`, otimizado para consumo imediato por qualquer ferramenta de BI ou Machine Learning.

1.4. Planejamento do Projeto

O projeto utiliza o framework CRISP-DM para garantir que a engenharia de dados seja rigorosa, permitindo que futuras campanhas (Peças, Serviços, Pneus, etc.) utilizem este mesmo motor de processamento como sua base oficial de faturamento.

2. Entendimento dos Dados

Nesta fase, é analisado a estrutura e na qualidade das bases transacionais. O objetivo é validar a integridade do faturamento e identificar anomalias sistêmicas antes de qualquer cálculo de ROI para a campanha de lubrificantes.

Dicionário de Dados: Vendas (df_vendas)

Atributo	Descrição
Cód. Produto	Código identificador da peça ou serviço John Deere.
Descrição	Nome do item vendido.
Subgrupo	Categoria secundária do produto.
Grupo	Categoria principal.
Nota Fiscal	Número do documento fiscal (faturamento).
Filial	Unidade INOVA responsável pela venda.
Centro Custo	Centro de custo associado à venda.
Descrição CC	Nome do centro de custo da operação.
Data Emissão	Data em que a nota de venda foi gerada.
Consultor	Consultor que realizou a venda.
CNPJ	Cadastro Nacional da Pessoa Jurídica do cliente.
Cliente	Razão Social ou nome do comprador.
Valor Bruto	Valor total da venda com impostos.
Valor Líquido	Valor efetivo de receita da venda.
Impostos	Valor total de carga tributária da nota.
Custo	Custo de mercadoria (CMV) para cálculo de margem.
Desconto	Abatimento financeiro aplicado no item.
Qtd.	Quantidade de unidades vendidas.
ML R\$	Margem Líquida em valor monetário.
ML %	Margem Líquida em percentual.

Dicionário de Dados: Devoluções (df_devolucoes)

Atributo	Descrição
Cód. Produto	Código identificador da peça ou serviço John Deere.
Descrição	Nome do item vendido.
Subgrupo	Categoria secundária do produto.
Grupo	Categoria principal.
Nota Fiscal	Número da nota de devolução.
Filial	Unidade INOVA responsável pela devolução.
Centro Custo	Centro de custo associado à devolução.
Descrição CC	Nome do centro de custo da operação.
Data	Data do registro da devolução.
Consultor	Consultor que realizou a venda.
CNPJ	Cadastro Nacional da Pessoa Jurídica do cliente.
Cliente	Razão Social do cliente.
Valor Bruto	Valor total da nota de devolução.
Valor Dev.	Valor líquido que será estornado/devolvido.
Imposto	Impostos incidentes sobre a devolução.
Custo	Estorno do custo da mercadoria.
Desconto	Desconto que havia sido aplicado na venda original.
Qtd.	Quantidade de unidades retornadas.
ML R\$	Margem Líquida em valor monetário.
ML %	Margem Líquida em percentual.

3. Análise Exploratória

3.1. Ingestão e Auditoria de Observabilidade (Data Gaps)

Nesta etapa inicial, o script realiza a orquestração e ingestão direta dos arquivos brutos extraídos do Dashboard "Diário de Bordo" do Power BI em formato `.xlsx`. A nova arquitetura elimina a necessidade de conversões manuais externas, consolidando os dados em memória para processamento imediato.

Protocolo de Ingestão e Qualidade:

- **Consolidação Multi-Arquivo:** Unificação dos históricos de vendas (`2022_23` , `2024` , `2025` e `2026`) em um único motor de processamento, abrangendo o

período de **setembro/2022 a março/2026**.

- **Deduplicação Nativa:** Remoção de registros 100% idênticos no momento da carga, mitigando erros de sobreposição entre extrações anuais.
- **Auditoria de Continuidade (Data Gaps):** Implementação de um monitor de saúde que valida o intervalo entre arquivos. Foi configurada uma tolerância de **4 dias** para acomodar feriados prolongados e finais de semana, garantindo que não existam lacunas ("buracos") de dados no histórico.
- **Monitoramento de Freshness:** Validação em tempo real da defasagem entre a última venda registrada e a data atual, garantindo a atualização da base para a Campanha de Lubrificantes.

Bases Processadas:

- `df_vendas` : Registros de vendas.
- `df_devolucoes` : Registros de estornos e devoluções.

Valor Estratégico: Este controle de observabilidade impede que períodos sem extração sejam interpretados erroneamente como quedas de mercado ou churn, garantindo que o "Dataset de Ouro" seja contínuo, íntegro e confiável para as análises subsequentes.

```
In [30]: # --- 3.1. Ingestão, Consolidação e Auditoria de Continuidade (Calibrada) ---
import pandas as pd
import os
import warnings
from datetime import datetime

warnings.filterwarnings('ignore', category=UserWarning, module='openpyxl')

diretorio = r'C:\Projetos\Inova\campanha_lubrificantes'
arquivos_vendas = [
    'Detalhamento_Vendas_22_23.xlsx',
    'Detalhamento_Vendas_24.xlsx',
    'Detalhamento_Vendas_25.xlsx',
    'Detalhamento_Vendas_26.xlsx'
]

lista_vendas = []
relatorio_datas = []

print("--- 🚀 Iniciando Ingestão e Auditoria de Gaps (Vendas) ---")

# 1. Processamento de Vendas
for arq in arquivos_vendas:
    caminho = os.path.join(diretorio, arq)
    if os.path.exists(caminho):
        df_temp = pd.read_excel(caminho, engine='openpyxl')
        df_temp['Data Emissão'] = pd.to_datetime(df_temp['Data Emissão'])

        d_min = df_temp['Data Emissão'].min()
        d_max = df_temp['Data Emissão'].max()

        relatorio_datas.append({
            'Arquivo': arq,
```

```

        'Min': d_min,
        'Max': d_max,
        'Linhas': len(df_temp)
    })

    lista_vendas.append(df_temp)
    print(f"✓ {arq}: {len(df_temp):,} linhas ({d_min.strftime('%d/%m/%Y')} :
else:
    print(f"✗ ALERTA: Arquivo {arq} NÃO localizado!")

# Consolidação e Deduplicação
df_vendas_bruto = pd.concat(lista_vendas, ignore_index=True)
df_vendas = df_vendas_bruto.drop_duplicates().reset_index(drop=True)

# 2. Processamento de Devoluções
caminho_dev = os.path.join(diretorio, 'Detalhamento_Devolucoes.xlsx')
if os.path.exists(caminho_dev):
    df_dev_bruto = pd.read_excel(caminho_dev, engine='openpyxl')
    df_dev_bruto['Data'] = pd.to_datetime(df_dev_bruto['Data'])
    df_devolucoes = df_dev_bruto.drop_duplicates().reset_index(drop=True)
    print(f"\n✓ Devoluções: {len(df_devolucoes):,} linhas lidas.")
else:
    df_devolucoes = pd.DataFrame()

# 3. Análise de Continuidade com Tolerância para Feriados
df_auditoria = pd.DataFrame(relatorio_dados).sort_values('Min')
df_auditoria['Gap_Proximo'] = (df_auditoria['Min'].shift(-1) - df_auditoria['Max

# 4. Relatório de Observabilidade Final
print("\n" + "="*80)
print(f"{'RELATÓRIO DE SAÚDE DOS DADOS (OBSERVABILITY)':^80}")
print("="*80)

hoje = datetime.now()
ultima_venda = df_vendas['Data Emissão'].max()
atraso_extracao = (hoje - ultima_venda).days

for i, row in df_auditoria.iterrows():
    gap = row['Gap_Proximo']
    # AJUSTE: Tolerância de 4 dias para cobrir feriados prolongados (ex: Ano Nov
    status = "✓ CONTINUIDADE OK" if (gap <= 4 or pd.isna(gap)) else f"⚠ GAP D
    print(f"Arquivo: {row['Arquivo']:<35} | Status: {status}")

print("-" * 80)
# Para 2026, mantemos um alerta se a extração tiver mais de 5 dias de atraso
if atraso_extracao > 5:
    print(f"⚠ ALERTA DE ATUALIZAÇÃO: Última venda registrada há {atraso_extrac
else:
    print(f"✓ ATUALIZAÇÃO: Extração de 2026 está em dia (Defasagem: {atraso_ex
print("="*80)

```

```

--- 📁 Iniciando Ingestão e Auditoria de Gaps (Vendas) ---
✓ Detalhamento_Vendas_22_23.xlsx: 146,122 linhas (05/09/2022 a 29/12/2023)
✓ Detalhamento_Vendas_24.xlsx: 125,767 linhas (02/01/2024 a 31/12/2024)
✓ Detalhamento_Vendas_25.xlsx: 138,114 linhas (02/01/2025 a 31/12/2025)
✓ Detalhamento_Vendas_26.xlsx: 22,951 linhas (02/01/2026 a 10/03/2026)

✓ Devoluções: 8,021 linhas lidas.

```

RELATÓRIO DE SAÚDE DOS DADOS (OBSERVABILITY)

```

Arquivo: Detalhamento_Vendas_22_23.xlsx | Status: ✅ CONTINUIDADE OK
Arquivo: Detalhamento_Vendas_24.xlsx | Status: ✅ CONTINUIDADE OK
Arquivo: Detalhamento_Vendas_25.xlsx | Status: ✅ CONTINUIDADE OK
Arquivo: Detalhamento_Vendas_26.xlsx | Status: ✅ CONTINUIDADE OK

```

```

✅ ATUALIZAÇÃO: Extração de 2026 está em dia (Defasagem: 3 dias).

```

```

In [31]: # --- 3.1. Configuração de Execução e Ingestão de Dados ---
import pandas as pd
import os
import warnings
from datetime import datetime

warnings.filterwarnings('ignore', category=UserWarning, module='openpyxl')

# =====
# PARÂMETRO DE CONTROLE:
# True = Lê os arquivos .xlsx (Lento, use para atualizar a base)
# False = Lê os arquivos .parquet de cache (Rápido, use para manutenção)
MODO_ATUALIZAR_DADOS = False
# =====

diretorio = r'C:\Projetos\Inova\campanha_lubrificantes'
arquivos_vendas_config = [
    'Detalhamento_Vendas_22_23.xlsx',
    'Detalhamento_Vendas_24.xlsx',
    'Detalhamento_Vendas_25.xlsx',
    'Detalhamento_Vendas_26.xlsx'
]

cache_vendas = os.path.join(diretorio, 'cache_vendas_consolidado.parquet')
cache_devolucoes = os.path.join(diretorio, 'cache_devolucoes_consolidado.parquet')

if MODO_ATUALIZAR_DADOS:
    print("--- 📁 MODO ATUALIZAÇÃO: Processando Origens XLSX ---")
    lista_vendas = []
    relatorio_dados = []

    # 1. Processamento de Vendas (Excel)
    for arq in arquivos_vendas_config:
        caminho = os.path.join(diretorio, arq)
        if os.path.exists(caminho):
            df_temp = pd.read_excel(caminho, engine='openpyxl')
            df_temp['Data Emissão'] = pd.to_datetime(df_temp['Data Emissão'])

            relatorio_dados.append({
                'Arquivo': arq, 'Min': df_temp['Data Emissão'].min(),
                'Max': df_temp['Data Emissão'].max(), 'Linhas': len(df_temp)
            })

```

```

    })
    lista_vendas.append(df_temp)
    print(f"✓ {arq} lido.")
else:
    print(f"✗ ALERTA: {arq} NÃO localizado!")

df_vendas = pd.concat(lista_vendas, ignore_index=True).drop_duplicates().reset_index()

# 2. Processamento de Devoluções (Excel)
caminho_dev = os.path.join(diretorio, 'Detalhamento_Devolucoes.xlsx')
if os.path.exists(caminho_dev):
    df_devolucoes = pd.read_excel(caminho_dev, engine='openpyxl').drop_duplicates()
    df_devolucoes['Data'] = pd.to_datetime(df_devolucoes['Data'])
    print(f"✓ Devoluções lidas.")

# 3. Salvando Cache para a próxima manutenção
df_vendas.to_parquet(cache_vendas, compression='snappy')
df_devolucoes.to_parquet(cache_devolucoes, compression='snappy')
print(f"💾 Checkpoint salvo em Parquet.")

else:
    print(f"--- ⚡ MODO MANUTENÇÃO: Carregando Checkpoint via Parquet ---")
    df_vendas = pd.read_parquet(cache_vendas)
    df_devolucoes = pd.read_parquet(cache_devolucoes)

# Gera o relatório de datas para a auditoria a partir do cache
# (Simulamos o relatório apenas para manter a auditoria funcional no modo rá
relatorio_datas = [{'Arquivo': 'CACHE_CONSOLIDADO', 'Min': df_vendas['Data Emissã
                    'Max': df_vendas['Data Emissão'].max(), 'Linhas': len(df_vendas)}]

# =====
# 4. AUDITORIA DE CONTINUIDADE (Sempre Executada)
# =====
print("\n" + "="*80)
print(f"{'RELATÓRIO DE SAÚDE E CONTINUIDADE DOS DADOS':^80}")
print("="*80)

if MODO_ATUALIZAR_DADOS:
    df_auditoria = pd.DataFrame(relatorio_datas).sort_values('Min')
    df_auditoria['Gap_Proximo'] = (df_auditoria['Min'].shift(-1) - df_auditoria['Max']).dt.days

    for i, row in df_auditoria.iterrows():
        gap = row['Gap_Proximo']
        status = "✅ OK" if (gap <= 4 or pd.isna(gap)) else f"⚠️ GAP: {int(gap)} dias"
        print(f"Arquivo: {row['Arquivo'][:35]} | Período: {row['Min'].strftime('%d/%m/%Y')} - {row['Max'].strftime('%d/%m/%Y')} | Status: {status}")

hoje = datetime.now()
ultima_venda = df_vendas['Data Emissão'].max()
atraso = (hoje - ultima_venda).days

print("-" * 80)
print(f"Status Atualização 2026: Última venda em {ultima_venda.strftime('%d/%m/%Y')}")
print(f"Total df_vendas: {len(df_vendas):,} linhas | Total df_devolucoes: {len(df_devolucoes):,} linhas")
print("="*80)

```

--- ⚡ MODO MANUTENÇÃO: Carregando Checkpoint via Parquet ---

```
=====
RELATÓRIO DE SAÚDE E CONTINUIDADE DOS DADOS
=====
-----
Status Atualização 2026: Última venda em 10/03/2026 (3 dias atrás).
Total df_vendas: 432,951 linhas | Total df_devolucoes: 8,021 linhas.
=====
```

```
=====
RELATÓRIO DE SAÚDE E CONTINUIDADE DOS DADOS
=====
-----
Status Atualização 2026: Última venda em 10/03/2026 (3 dias atrás).
Total df_vendas: 432,951 linhas | Total df_devolucoes: 8,021 linhas.
=====
```

```
In [32]: import pandas as pd

# 1. Importação dos arquivos Parquet
df_vendas = pd.read_parquet('Detalhamento_Vendas.parquet')
df_devolucoes = pd.read_parquet('Detalhamento_Devolucoes.parquet')

# 2. Detalhamento Simples do que foi importado
def detalhar_base(df, nome):
    print(f"--- Base: {nome} ---")
    print(f"Dimensões: {df.shape[0]} linhas x {df.shape[1]} colunas")
    print(f"Período: {df.select_dtypes(include=['datetime64']).iloc[:,0].min()}")
    print(f"Colunas Disponíveis: {df.columns.tolist()}\n")

detalhar_base(df_vendas, "Vendas")
detalhar_base(df_devolucoes, "Devoluções")
```

```
--- Base: Vendas ---
Dimensões: 432951 linhas x 21 colunas
Período: 2022-09-05 00:00:00 até 2026-03-10 00:00:00
Colunas Disponíveis: ['Cód. Produto', 'Descrição', 'Subgrupo', 'Grupo', 'Nota Fis
cal', 'Filial', 'Centro Custo', 'Descrição CC', 'Data Emissão', 'Consultor', 'Cód. Client
e', 'CNPJ', 'Cliente', 'Valor Bruto', 'Valor Líquido', 'Impostos', 'Custo', 'Desconto', 'Qtd.', 'ML R$', 'ML %']
```

```
--- Base: Devoluções ---
Dimensões: 8021 linhas x 21 colunas
Período: 2022-09-15 00:00:00 até 2026-03-04 00:00:00
Colunas Disponíveis: ['Cód. Produto', 'Descrição', 'Subgrupo', 'Grupo', 'Nota Fis
cal', 'Filial', 'Centro Custo', 'Descrição CC', 'Data', 'Consultor', 'Cód. Client
e', 'CNPJ', 'Cliente', 'Valor Bruto', 'Valor Dev.', 'Imposto', 'Custo', 'Descont
o', 'Qtd.', 'ML $', 'ML %']
```

3.2. Dicionário de Tradução/Padronização

```
In [33]: # 3.2. Dicionário de Tradução/Padronização
# Mapeamos o nome atual para o padrão 'snake_case'
map_vendas = {
    'Cód. Produto': 'cod_produto',
    'Descrição': 'descricao',
    'Subgrupo': 'subgrupo',
```



```

    'Grupo': 'grupo',
    'Nota Fiscal': 'nota_fiscal',
    'Filial': 'filial',
    'Centro Custo': 'centro_custo',
    'Descrição CC': 'descricao_cc',
    'Data Emissão': 'data_emissao',
    'Consultor': 'consultor',
    'Cód. Cliente': 'cod_cliente',
    'CNPJ': 'cnpj',
    'Cliente': 'cliente',
    'Valor Bruto': 'valor_bruto',
    'Valor Líquido': 'valor_liquido',
    'Impostos': 'impostos',
    'Custo': 'custo',
    'Desconto': 'desconto',
    'Qtd.': 'quantidade',
    'ML R$': 'margem_bruta_valor',
    'ML %': 'margem_bruta_perc'
}

map_devolucoes = {
    'Cód. Produto': 'cod_produto',
    'Descrição': 'descricao',
    'Subgrupo': 'subgrupo',
    'Grupo': 'grupo',
    'Nota Fiscal': 'nota_fiscal',
    'Filial': 'filial',
    'Centro Custo': 'centro_custo',
    'Descrição CC': 'descricao_cc',
    'Data': 'data_emissao',          # Unificando com Vendas
    'Consultor': 'consultor',
    'Cód. Cliente': 'cod_cliente',
    'CNPJ': 'cnpj',
    'Cliente': 'cliente',
    'Valor Bruto': 'valor_bruto_dev',
    'Valor Dev.': 'valor_liquido',
    'Imposto': 'impostos',          # Unificando plural
    'Custo': 'custo',
    'Desconto': 'desconto',
    'Qtd.': 'quantidade',
    'ML $': 'margem_bruta_valor',   # Unificando símbolo
    'ML %': 'margem_bruta_perc'
}

# 2. Aplicação do Rename
df_vendas.rename(columns=map_vendas, inplace=True)
df_devolucoes.rename(columns=map_devolucoes, inplace=True)

print("✅ Colunas padronizadas.")

```

✅ Colunas padronizadas.

3.3. Auditoria de Estrutura Pós-Padronização

Após a aplicação dos dicionários de tradução (`map_vendas` e `map_devolucoes`), executamos uma auditoria para validar a consistência das dimensões e a correta aplicação da nomenclatura *snake_case*.

- **Objetivo:** Garantir que o volume de registros (linhas) e a quantidade de atributos (colunas) estejam íntegros antes das operações de limpeza de tipos e valores nulos.

```
In [34]: # 3.3. Checkpoint de Integridade - Auditoria de Estrutura

# Lista de datasets e seus títulos para o relatório
datasets = [df_vendas, df_devolucoes]
titles = ["df_vendas", "df_devolucoes"]

# Construção do DataFrame de Auditoria
info_df = pd.DataFrame({})
info_df['dataset'] = titles
info_df['qtd_colunas'] = [len(df.columns) for df in datasets]
info_df['nome_colunas'] = [' '.join(list(df.columns)) for df in datasets]
info_df['linhas_colunas'] = [len(df) for df in datasets]

# Exibição com gradiente para facilitar a leitura visual
info_df.style.background_gradient(cmap='Greys')
```

```
Out[34]:
```

	dataset	qtd_colunas	nome_colunas	linhas_colunas
0	df_vendas	21	cod_produto, descricao, subgrupo, grupo, nota_fiscal, filial, centro_custo, descricao_cc, data_emissao, consultor, cod_cliente, cnpj, cliente, valor_bruto, valor_liquido, impostos, custo, desconto, quantidade, margem_bruta_valor, margem_bruta_perc	432951
1	df_devolucoes	21	cod_produto, descricao, subgrupo, grupo, nota_fiscal, filial, centro_custo, descricao_cc, data_emissao, consultor, cod_cliente, cnpj, cliente, valor_bruto_dev, valor_liquido, impostos, custo, desconto, quantidade, margem_bruta_valor, margem_bruta_perc	8021

Observação:

- Dataset com maior número de linhas é o df_vendas

3.4. Auditoria de Tipagem (Dtypes Audit)

Separação das colunas em três grupos fundamentais. Essa taxonomia permite definir as técnicas de EDA que usaremos a seguir:

1. **Colunas Numéricas (Quantitativas):** Utilizadas para cálculos de faturamento líquido, margem de contribuição (ML) e descontos.
2. **Colunas de Data (Temporais):** Essenciais para identificar a "Recência" do cliente (RFM) e janelas de Churn na Linha Amarela.
3. **Colunas de Objeto (Qualitativas/Categoriais):** Contêm metadados como CNPJ, Descrição de Peças e Segmentação de Consultores.

Resultado da Auditoria

- **Consistência Identificada:** Ambas as bases possuem a mesma largura de colunas (21), facilitando operações de `concat` se necessário no futuro.
- **Validação de Chave:** Confirmamos que `cnnpj` e `cod_cliente` estão no grupo numérico (`float64`), o que reforça a necessidade de tratamento de tipos na Fase 3 para evitar perda de integridade.

```
In [35]: # 3.4. Auditoria de Tipagem (Dtypes Audit)

import pandas as pd

# 1. Lista de datasets e seus títulos
datasets = [df_vendas, df_devolucoes]
titles = ["Vendas", "Devoluções"]

# 2. Definição dos tipos para auditoria
numerics = ['int16', 'int32', 'int64', 'float16', 'float32', 'float64']
dates = ['datetime64', 'datetime64[ns]']

# 3. Construção do DataFrame de Auditoria de Tipos
df_types_check = pd.DataFrame({'dataset': titles})

# Colunas Numéricas
df_types_check['numeric_count'] = [len(df.select_dtypes(include=numerics).columns) for df in datasets]
df_types_check['numeric_cols'] = [', '.join(df.select_dtypes(include=numerics).columns) for df in datasets]

# Colunas de Data
df_types_check['date_count'] = [len(df.select_dtypes(include=dates).columns) for df in datasets]
df_types_check['date_cols'] = [', '.join(df.select_dtypes(include=dates).columns) for df in datasets]

# Colunas de Objeto (Texto/Categorias)
df_types_check['object_count'] = [len(df.select_dtypes(include='object').columns) for df in datasets]
df_types_check['object_cols'] = [', '.join(df.select_dtypes(include='object').columns) for df in datasets]

# 4. Exibição com Estilização para facilitar a leitura técnica
display(df_types_check.style.background_gradient(cmap='Greens', subset=['numeric_count', 'date_count', 'object_count']))
```

	dataset	numeric_count	numeric_cols	date_count	date_cols	object_count
0	Vendas	12	nota_fiscal, centro_custo, cod_cliente, cnpj, valor_bruto, valor_liquido, impostos, custo, desconto, quantidade, margem_bruta_valor, margem_bruta_perc	1	data_emissao	8
1	Devoluções	12	nota_fiscal, centro_custo, cod_cliente, cnpj, valor_bruto_dev, valor_liquido, impostos, custo, desconto, quantidade, margem_bruta_valor, margem_bruta_perc	1	data_emissao	8

- **Observações:**
- **Colunas que devem ser alteradas:** `nota_fiscal`, `centro_custo`, `cod_cliente` e `cnpj`.
- **Ação Técnica:** Conversão de numérico para `string` (object), com remoção de sufixos decimais (`.0`)
- **Objetivo:** Isolar estas colunas da análise estatística (média/mediana) e garantir a integridade da chave primária para o cruzamento com a base de Potencial.

3.4.1. Refinamento de Tipagem (Casting de IDs)

```
In [36]: ### 3.4.1. Refinamento de Tipagem (Casting de IDs)

import pandas as pd

# 1. Definição das colunas que devem ser tratadas como ID (Objeto)
cols_to_object = ['nota_fiscal', 'centro_custo', 'cod_cliente', 'cnpj']

# 2. Aplicação da conversão em ambos os datasets
for df in [df_vendas, df_devolucoes]:
    for col in cols_to_object:
        if col in df.columns:
            # Primeiro removemos o .0 (caso exista por ser float) e convertemos
            df[col] = df[col].astype(str).str.replace('\.0$', '', regex=True)
            # Substituímos 'nan' textual (gerado pelo astype em nulos) por None
            df[col] = df[col].replace('nan', None)

print("✅ Tipagem atualizada com sucesso.")

# 3. Verificação rápida para confirmar se saíram do grupo numérico
```

```
print("\n--- Novos tipos em Vendas ---")
print(df_vendas[cols_to_object].dtypes)
```

✓ Tipagem atualizada com sucesso.

```
--- Novos tipos em Vendas ---
nota_fiscal      object
centro_custo      object
cod_cliente      object
cnpj              object
dtype: object
```

3.4.2. Diagnóstico de Colunas Numéricas

A exploração estatística revelou outliers extremos que indicam erros de integração ou unidades de medida incorretas no ERP:

1. **Outliers de Faturamento:** O `valor_liquido` máximo é provavelmente um erro de sistema ou um lançamento de ajuste contábil.
2. **Erros de Quantidade:** A `quantidade` máxima de unidades em um único registro sugere erro de unidade de medida.
3. **Margens Impossíveis:** * Temos `margem_bruta_perc` mínima indica que o custo registrado é absurdamente maior que a venda, ou um erro de digitação.
 - A `margem_bruta_valor` mínima em uma linha de venda sugere prejuízo operacional grave ou erro de custo.

In [37]: *### 3.4.2. Diagnóstico de Colunas Numéricas*

```
import pandas as pd

# 1. Seleção de métricas de interesse (usando os nomes padronizados)
metrics_vendas = ['nota_fiscal', 'valor_bruto', 'valor_liquido', 'impostos', 'cu
metrics_dev = ['nota_fiscal', 'valor_bruto_dev', 'valor_liquido', 'impostos', 'c

def gerar_tabela_estadistica(df, title, columns):
    print(f"--- 📊 Tabela de Auditoria Estatística: {title} ---")

    # Filtra apenas as colunas que existem no DF para evitar erro
    cols_existentes = [c for c in columns if c in df.columns]

    # Geração das estatísticas descritivas
    stats = df[cols_existentes].describe().T

    # Cálculo do Coeficiente de Variação (CV)
    # CV > 1 indica alta dispersão
    stats['cv'] = stats['std'] / stats['mean']

    # Exibição estilizada com separador de milhar e 2 casas decimais
    display(stats.style.format("{:,.2f}").background_gradient(cmap='Greens', sub

# Execução
gerar_tabela_estadistica(df_vendas, "Vendas", metrics_vendas)
gerar_tabela_estadistica(df_devolucoes, "Devoluções", metrics_dev)
```

--- 📊 Tabela de Auditoria Estatística: Vendas ---

	count	mean	std	min	25%	50%	75%
valor_bruto	432,946.00	3,190.14	577,694.32	0.01	111.08	318.00	875.60
valor_liquido	432,946.00	3,017.94	546,712.92	0.01	102.78	289.39	802.59
impostos	432,946.00	172.20	31,060.99	0.00	0.00	14.60	56.62
custo	432,946.00	2,128.06	385,123.55	0.00	62.47	184.84	543.46
desconto	432,946.00	222.43	39,881.30	0.00	0.00	0.00	46.27
quantidade	432,946.00	8.66	1,562.36	0.70	1.00	1.00	2.00
margem_bruta_valor	432,946.00	889.89	161,709.34	-234,868.06	23.53	90.72	256.77
margem_bruta_perc	432,946.00	0.28	25.04	-16,296.00	0.26	0.35	0.42

--- 📊 Tabela de Auditoria Estatística: Devoluções ---

	count	mean	std	min	25%	50%	75%
valor_bruto_dev	8,019.00	8,105.30	365,120.34	0.83	165.66	435.05	1,350.15
valor_liquido	8,019.00	7,540.09	339,904.71	0.71	147.80	392.58	1,210.59
impostos	8,019.00	202.67	9,097.60	-172.81	0.00	12.76	51.47
custo	8,019.00	5,717.41	258,492.27	0.00	84.27	225.82	737.17
desconto	8,019.00	588.06	27,064.14	0.00	0.00	1.17	79.84
quantidade	8,019.00	8.40	376.83	1.00	1.00	1.00	2.00
margem_bruta_valor	8,019.00	2,185.22	98,239.09	-82,627.87	56.30	176.93	497.53
margem_bruta_perc	8,019.00	0.32	6.67	-590.92	0.36	0.43	0.49

3.4.2.1. Descoberta de Erros Estruturais (Linhas de Totalização)

```
In [38]: ### 3.4.2.1. Descoberta de Erros Estruturais (Linhas de Totalização)

import pandas as pd

# 1. Identificando as 10 maiores transações com todas as colunas
outliers_faturamento = df_vendas.nlargest(20, 'valor_liquido')

print("--- 🚨 Top 10 Transações Suspeitas (Visão Completa) ---")
# display(outliers_faturamento) exibirá todas as colunas no Jupyter
display(outliers_faturamento)

# 2. Identificando Margens Irreais com Visão Completa
# Filtro: Margens menores que -100% ou maiores que 100%
margens_suspeitas = df_vendas[(df_vendas['margem_bruta_perc'] < -1) | (df_vendas

print(f"\n⚠️ Total de registros com margens irreais: {len(margens_suspeitas)}")

if len(margens_suspeitas) > 0:
    print("--- Amostra de Registros com Margens Inconsistentes ---")
```

```
# Exibindo os 10 casos mais críticos de margem negativa  
display(margens_suspeitas.sort_values('margem_bruta_perc').head(10))
```

--- 🚨 Top 10 Transações Suspeitas (Visão Completa) ---

	cod_produto		descricao	subgrupo	grupo	nota_fiscal	filial	cen
146119	Total		None	None	None	None	None	
409999	Total		None	None	None	None	None	
271886	Total		None	None	None	None	None	
432949	Total		None	None	None	None	None	
13506	None		None	None	None	9	0201 - Contagem	
13542	None		None	None	None	111419	0201 - Contagem	
13524	None		None	None	None	30	0201 - Contagem	
209023	9299930	CONJ. PLANETARIOS	None	JDPC	154812		0212 - CSN	
13529	None		None	None	None	35	0201 - Contagem	
13530	None		None	None	None	36	0201 - Contagem	
274363	RE566098	MOTOR A DIESEL	None	JDPC	193800		0212 - CSN	
13528	None		None	None	None	34	0201 - Contagem	
13543	VEIC_016303	1F9210GXVND523077	None	0098	41		0201 - Contagem	
13544	VEIC_016304	1F9210GXAND523078	None	IMO1	40		0201 - Contagem	
13545	VEIC_016309	1F9210GXVND523080	None	IMO1	39		0201 - Contagem	
235910	2112478	FERRAMENTA DE FRESAGEM	None	CBPC	399		0210 - Pouso Alegre	
200810	None		None	None	None	174909	0201 - Contagem	

	cod_produto	descricao	subgrupo	grupo	nota_fiscal	filial	cen
	13507	None	None	None	10	0201 - Contagem	
	13508	None	None	None	11	0201 - Contagem	
	63702	FYB60003523	MOTOR	None	JDPC	114677	0212 - CSN

20 rows × 21 columns

⚠ Total de registros com margens irreais: 769
--- Amostra de Registros com Margens Inconsistentes ---

	cod_produto	descricao	subgrupo	grupo	nota_fiscal	filial	centr
155562	22AC16224	COPO TRMICO JOHN DEERE 400 ML - PRETO	None	JDCO	159528	0201 - Contagem	
5705	CQM20196	AIR CLEAN JOHN DEERE HIGIENIZA	None	CLGD	123201	0201 - Contagem	
19386	CQM20196	AIR CLEAN JOHN DEERE HIGIENIZA	None	CLGD	122196	0201 - Contagem	
5704	CQM20196	AIR CLEAN JOHN DEERE HIGIENIZA	None	CLGD	122823	0201 - Contagem	
136968	CQM20196	AIR CLEAN JOHN DEERE HIGIENIZA	None	CLGD	2226	0204 - Urberlândia	
347202	CQM20131	OLEO HY- GARD 1000L TRANSMISSAO	LUBRIFICANTE	LUB	16635	0204 - Urberlândia	
153023	AT340027P	MANG MONTADA COLETA TANQUE HIDRAULICO - MANG M...	None	JDOU	196	0212 - CSN	
17843	AT318789	KIT RETENTOR	None	JDPC	133895	0201 - Contagem	
398453	T187987	FACA CORTE DURA-MAX	FPS	JDPC	212935	0212 - CSN	
398452	T187987	FACA CORTE DURA-MAX	FPS	JDPC	212933	0212 - CSN	

10 rows × 21 columns



3.4.2.2. Remoção de Linhas de Totalização (Outliers do PowerBI)

```
In [39]: ##### 3.4.2.2. Remoção de Linhas de Totalização (Outliers do PowerBI)
# Preservando a base original de 346.622 registros
df_vendas_clean = df_vendas.copy()
df_dev_clean = df_devolucoes.copy()

# Auditoria Financeira Antes da Remoção
valor_total_antes = df_vendas_clean['valor_liquido'].sum()
vendas_antes = len(df_vendas_clean)
```

```
# Essas linhas representam somatórios do sistema e inflam artificialmente a base
df_vendas_clean = df_vendas_clean[df_vendas_clean['cod_produto'] != 'Total']
df_dev_clean = df_dev_clean[df_dev_clean['cod_produto'] != 'Total']

# Auditoria Financeira Depois da Remoção
valor_total_depois = df_vendas_clean['valor_liquido'].sum()
vendas_depois = len(df_vendas_clean)
linhas_removidas_total = vendas_antes - vendas_depois

print(f"--- Relatório de Auditoria: Remoção de Totalizações ---")
print(f"✅ Linhas removidas: {linhas_removidas_total}")
print(f"💰 Faturamento Total ANTES: R$ {valor_total_antes:,.2f}")
print(f"💰 Faturamento Total DEPOIS: R$ {valor_total_depois:,.2f}")
print(f"📉 Impacto financeiro (expurgo): R$ {(valor_total_antes - valor_total_depois):,.2f}")
print(f"-----")
```

```
--- Relatório de Auditoria: Remoção de Totalizações ---
```

```
✅ Linhas removidas: 4
```

```
💰 Faturamento Total ANTES: R$ 1,306,606,509.66
```

```
💰 Faturamento Total DEPOIS: R$ 653,301,131.96
```

```
📉 Impacto financeiro (expurgo): R$ 653,305,377.70
```

3.4.2.3. Auditoria e Recálculo de Margem Bruta (Sanitização Financeira)

Após o diagnóstico de qualidade, identificamos que a base apresentava divergências entre o valor de margem registrado pelo sistema e a diferença aritmética entre receita e custo.

Justificativa Técnica:

- **Padronização de Conceito:** Embora a base original utilize a sigla "ML" (Margem Líquida), a ausência de despesas operacionais variáveis (frete, comissões) nos dados transacionais caracteriza o indicador como **Margem Bruta**.
- **Correção de Divergência:** O recálculo direto via `valor_liquido - custo` elimina inconsistências sistêmicas provenientes do ERP ou de arredondamentos na exportação do Power BI.

Lógica Aplicada:

1. **Margem Bruta Valor:** $Margem = ValorLiquido - Custo$.

2. **Margem Bruta %:** $Margem(\%) = \frac{Margem}{ValorLiquido}$ (com tratamento para divisão por zero).

In [40]: ##### 3.4.2.3. Auditoria e Recálculo de Margem Bruta (Sanitização Financeira)

```
# Justificativa: O diagnóstico revelou que 1,52% da base possuía inconsistências
# o lucro registrado e a diferença (Líquido - Custo). Recalculamos para garantir
# que o ROI da campanha de Lubrificantes seja preciso.
```

```
import numpy as np
```

```
def aplicar_margem_auditada(df):
```

```

"""
Recalcula a Margem Bruta para garantir consistência após a limpeza de
outliers e futuras inversões de sinal.
"""

# 1. Margem Bruta em Valor: Valor Líquido - Custo
# Tratamos nulos no custo como 0 para evitar NaNs no resultado financeiro
df['margem_bruta_valor'] = df['valor_liquido'] - df['custo'].fillna(0)

# 2. Margem Bruta Percentual: Margem / Valor Líquido
# np.where evita erro de divisão por zero caso o valor líquido seja 0
df['margem_bruta_perc'] = np.where(
    df['valor_liquido'] != 0,
    df['margem_bruta_valor'] / df['valor_liquido'],
    0
)

return df

# Aplicando nos dataframes que acabaram de passar pela remoção de totalizações
df_vendas_clean = aplicar_margem_auditada(df_vendas_clean)
df_dev_clean = aplicar_margem_auditada(df_dev_clean)

# Auditoria de Verificação Rápida
print(f"--- Relatório de Auditoria: Recálculo de Margens ---")
print(f"✅ Margem Bruta recalculada em {len(df_vendas_clean):,} linhas de Vendas")
print(f"✅ Margem Bruta recalculada em {len(df_dev_clean):,} linhas de Devoluções")
print(f"📊 Média de Margem Bruta (Vendas): {df_vendas_clean['margem_bruta_perc']}")
print(f"-----")

--- Relatório de Auditoria: Recálculo de Margens ---
✅ Margem Bruta recalculada em 432,947 linhas de Vendas.
✅ Margem Bruta recalculada em 8,020 linhas de Devoluções.
📊 Média de Margem Bruta (Vendas): 27.53%
-----

```

3.4.2.4. Arredondamento de Precisão Financeira

Para garantir a integridade da conciliação entre o Python e o ERP da INOVA, aplicamos uma camada de normalização decimal nas métricas monetárias.

Por que esta etapa é necessária?

- **Resíduos de Cálculo:** Operações de impostos, descontos e rateios de custo frequentemente geram dízimas infinitas que, se não tratadas, acumulam diferenças centesimais nos totais consolidados.
- **Alinhamento com o Power BI:** O arredondamento prévio no ETL evita discrepâncias visuais em cartões e tabelas dinâmicas no Dashboard.
- **Conformidade Fiscal:** Garante que os valores de `valor_liquido` e `impostos` reflitam a precisão de duas casas decimais utilizada na emissão das Notas Fiscais.

```

In [41]: # --- 3.4.2.4. Arredondamento de Precisão Financeira ---

# Definimos as Listas específicas para cada dataframe conforme o estado atual do
cols_vendas = [
    'valor_bruto', 'valor_liquido', 'impostos',
    'custo', 'desconto', 'margem_bruta_valor'
]

```

```

cols_devolucoes = [
    'valor_bruto_dev', 'valor_liquido', 'impostos',
    'custo', 'desconto', 'margem_bruta_valor'
]

# Aplicamos o arredondamento (round 2) para garantir precisão de centavos
df_vendas_clean[cols_vendas] = df_vendas_clean[cols_vendas].round(2)
df_dev_clean[cols_devolucoes] = df_dev_clean[cols_devolucoes].round(2)

# A coluna de percentual (margem_bruta_perc) costuma ser mantida com mais casas
# para precisão de KPI (ex: 4 casas), mas se desejar 2, descomente a linha abaixo
# df_vendas_clean['margem_bruta_perc'] = df_vendas_clean['margem_bruta_perc'].ro

print(f"✅ Arredondamento aplicado: {len(cols_vendas)} colunas em Vendas e {len(cols_devolucoes)} colunas em Devoluções.")
print(f"📊 Dados normalizados para evitar dízimas residuais no Master.")

```

✅ Arredondamento aplicado: 6 colunas em Vendas e 6 em Devoluções.

📊 Dados normalizados para evitar dízimas residuais no Master.

3.4.3. Diagnóstico de Volumetria Histórica

Para comparar grandezas em escalas distintas (Faturamento vs. Volume de Estornos), aplicamos a normalização **Min-Max**. Este método isola o valor absoluto e foca no **comportamento da curva**

Insight Esperado: Se a curva de devoluções (vermelha) estiver frequentemente acima da de vendas (verde) em termos de variação, temos um sinal de alerta no processo de expedição ou na qualidade das peças (Peças Genuínas vs. Erros de Catálogo).

```

In [42]: ### 3.4.3. Diagnóstico de Volumetria Histórica

import matplotlib.pyplot as plt
import pandas as pd
from sklearn.preprocessing import MinMaxScaler

# 1. Preparação dos Dados
# Filtramos 'Total' e garantimos que trabalhamos com cópias para evitar SettingWithCopyWarning
df_v_plot = df_vendas[df_vendas['cod_produto'] != 'Total'].copy()
df_d_plot = df_devolucoes[df_devolucoes['cod_produto'] != 'Total'].copy()

# 2. Agrupamento mensal usando 'ME' (Month End) conforme a nova documentação do pandas
# Nota: corrigido 'valor_liquido' para 'valor_liquido' na base de estornos
vendas_mes = df_v_plot.resample('ME', on='data_emissao')['valor_liquido'].sum()
dev_mes = df_d_plot.resample('ME', on='data_emissao')['valor_liquido'].sum()

# 3. Alinhamento e Normalização
df_timeline = pd.DataFrame({'vendas': vendas_mes, 'devolucoes': dev_mes}).fillna(0)

scaler = MinMaxScaler()
df_timeline_norm = pd.DataFrame(
    scaler.fit_transform(df_timeline),
    index=df_timeline.index,
    columns=df_timeline.columns
)

# 4. Plotagem
plt.figure(figsize=(14, 6))
plt.plot(df_timeline_norm.index, df_timeline_norm['vendas'], color='green', label='Vendas')
plt.plot(df_timeline_norm.index, df_timeline_norm['devolucoes'], color='red', label='Devoluções')

```

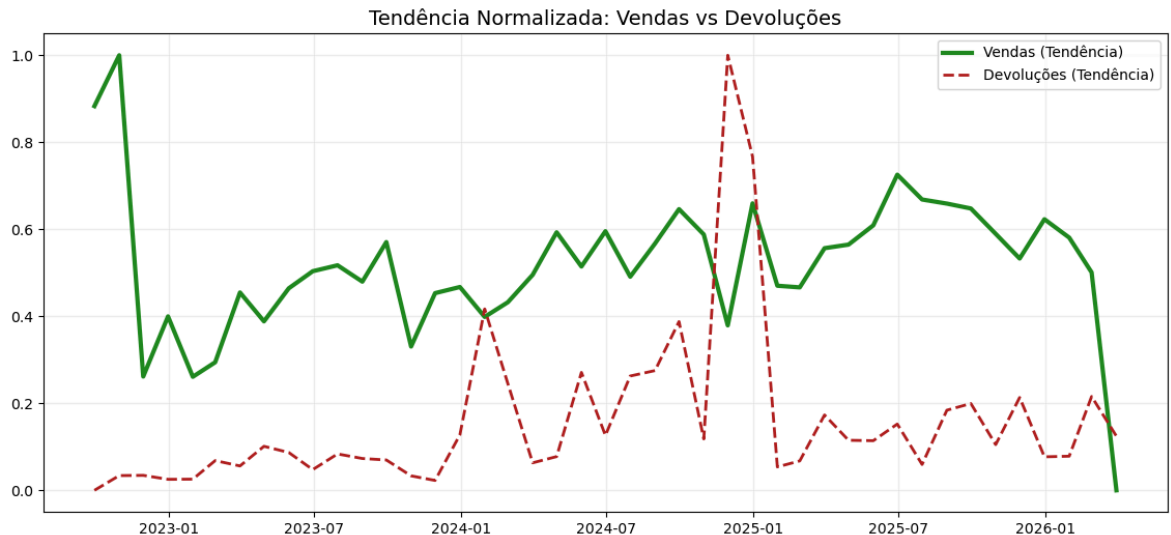
```

        label='Vendas (Tendência)', color='forestgreen', linewidth=3)
plt.plot(df_timeline_norm.index, df_timeline_norm['devolucoes'],
        label='Devoluções (Tendência)', color='firebrick', linestyle='--', line

plt.title('Tendência Normalizada: Vendas vs Devoluções', fontsize=14)
plt.legend()
plt.grid(True, alpha=0.2)
plt.show()

print("✅ Gráfico gerado. Observe se os picos de devolução coincidem com os meses de maior venda.")

```



✅ Gráfico gerado. Observe se os picos de devolução coincidem com os meses de maior venda.

3.4.4. Análise de Variáveis Categóricas (Cardinalidade e Perfil)

```

In [43]: ## 3.4.4. Análise de Variáveis Categóricas (Cardinalidade e Perfil)

import pandas as pd

# 1. Definição das colunas de objeto para análise (Incluindo 'cnpj' para auditoria)
cols_objeto = ['filial', 'grupo', 'subgrupo', 'consultor', 'cliente', 'cnpj', 'c']

def eda_categorica(df, title):
    print(f"--- 📁 Auditoria de Categorias: {title} ---")

    report_data = []
    total_linhas = len(df)

    for col in cols_objeto:
        if col in df.columns:
            # Calculando métricas de frequência e diversidade
            n_unique = df[col].nunique()
            counts = df[col].value_counts()

            if not counts.empty:
                top_val = counts.idxmax()
                top_freq = counts.max()
                perc_top = (top_freq / total_linhas) * 100

                report_data.append({
                    'Coluna': col,

```

```

        'Valores Únicos': n_unique,
        'Valor Mais Frequente': top_val,
        'Frequência': top_freq,
        '% de Concentração': f"{perc_top:.2f}%"
    })

df_report = pd.DataFrame(report_data)

# Estilização para destacar alta cardinalidade (azul)
# Importante para identificar colunas que podem ser chaves primárias ou nome
display(df_report.style.background_gradient(cmap='Blues', subset=['Valores Ú
    .set_caption(f"Análise de Cardinalidade - {title}"))

# Insight Proativo de Arquiteto para a Etapa 2.6.3
if 'cliente' in df.columns and 'cnpj' in df.columns:
    dif = df['cliente'].nunique() - df['cnpj'].nunique()
    if dif > 0:
        print(f"💡 Insight EDA: Existem {dif} variações de nomes a mais que
        print(f"Isso indica prováveis duplicidades de cadastro que serão san

print("-" * 60 + "\n")

# Execução nos datasets originais (Fase 2 - EDA)
eda_categorica(df_vendas, "Vendas")
eda_categorica(df_devolucoes, "Devoluções")

```

--- 🛒 Auditoria de Categorias: Vendas ---

Análise de Cardinalidade - Vendas

	Coluna	Valores Únicos	Valor Mais Frequente	Frequência	% de Concentração
0	filial	9	0201 - Contagem	188805	43.61%
1	grupo	31	JDPC	382270	88.29%
2	subgrupo	5	FILTROS	133553	30.85%
3	consultor	101	Lilian Queiroz	81365	18.79%
4	cliente	3971	CSN MINERACAO S.A.	106307	24.55%
5	cnpj	3988	8902291000468	69507	16.05%
6	cod_produto	20568	RE541922	10219	2.36%

--- 🛒 Auditoria de Categorias: Devoluções ---

Análise de Cardinalidade - Devoluções

	Coluna	Valores Únicos	Valor Mais Frequente	Frequência	% de Concentração
0	filial	7	0201 - Contagem	3282	40.92%
1	grupo	21	JDPC	7051	87.91%
2	subgrupo	5	FILTROS	2304	28.72%
3	consultor	70	Lilian Queiroz	922	11.49%
4	cliente	762	CSN MINERACAO S.A.	1412	17.60%
5	cnpj	762	8902291000468	1412	17.60%
6	cod_produto	3223	CQM20204	169	2.11%

3.5. Qualidade de Dados: Diagnóstico de Nulos

```
In [44]: # 3.5. Qualidade de Dados: Diagnóstico de Nulos

# Configuração dos datasets da INOVA
datasets = [df_vendas, df_devolucoes]
titles = ["Vendas", "Devoluções"]

info_df_n = pd.DataFrame({})
info_df_n['dataset'] = titles

# Criando coluna com o nome de todas as colunas do dataset
info_df_n['cols'] = [' ', '.join([col for col in df.columns]) for df in datasets]

# Total de colunas no dataset
info_df_n['cols_no'] = [df.shape[1] for df in datasets]

# Contagem total de células nulas no dataframe
info_df_n['null_no'] = [df.isnull().sum().sum() for df in datasets]

# Quantidade de colunas que possuem ao menos um valor nulo
info_df_n['null_cols_no'] = [len([col for col, null in df.isnull().sum().items()

# Nome das colunas que possuem valores nulos
info_df_n['null_cols'] = [' ', '.join([col for col, null in df.isnull().sum().item

# Exibição com gradiente visual
info_df_n.style.background_gradient(cmap='Greys')
```


Out[44]:

	dataset	cols	cols_no	null_no	null_cols_no	null_cols
0	Vendas	cod_produto, descricao, subgrupo, grupo, nota_fiscal, filial, centro_custo, descricao_cc, data_emissao, consultor, cod_cliente, cnpj, cliente, valor_bruto, valor_liquido, impostos, custo, desconto, quantidade, margem_bruta_valor, margem_bruta_perc	21	678183	21	cod_produto, descricao, subgrupo, grupo, nota_fiscal, filial, centro_custo, descricao_cc, data_emissao, consultor, cod_cliente, cnpj, cliente, valor_bruto, valor_liquido, impostos, custo, desconto, quantidade, margem_bruta_valor, margem_bruta_perc
1	Devoluções	cod_produto, descricao, subgrupo, grupo, nota_fiscal, filial, centro_custo, descricao_cc, data_emissao, consultor, cod_cliente, cnpj, cliente, valor_bruto_dev, valor_liquido, impostos, custo, desconto, quantidade, margem_bruta_valor, margem_bruta_perc	21	10070	21	cod_produto, descricao, subgrupo, grupo, nota_fiscal, filial, centro_custo, descricao_cc, data_emissao, consultor, cod_cliente, cnpj, cliente, valor_bruto_dev, valor_liquido, impostos, custo, desconto, quantidade, margem_bruta_valor, margem_bruta_perc

Observações:

A análise quantitativa de valores ausentes revelou pontos críticos que impactam diretamente a modelagem:

- **Saturação de Nulos em Vendas:** A base de Vendas apresenta **678.183 valores nulos** distribuídos por **todas as 21 colunas**. Isso indica que não temos apenas células vazias isoladas, mas provavelmente linhas inteiras sem dados (provenientes de quebras de página ou rodapés de totalização do Power BI).
- **Contaminação Generalizada:** O fato de `null_cols_no` ser igual a **21** (o total de colunas) em ambos os datasets confirma que todas as variáveis possuem ao menos um valor ausente. Isso reforça a necessidade de uma limpeza por linha (`dropna`) antes de qualquer análise estatística.
- **Proporcionalidade em Devoluções:** Embora a base de Devoluções tenha um volume menor de nulos (**10.070**), ela segue o mesmo padrão de contaminação em todas as colunas, sugerindo que o erro de exportação é sistemático no pipeline de dados.

- **Risco para Chave Primária:** Como `cod_produto` e `cnpj` apresentam nulos, qualquer tentativa de cruzamento (join) ou cálculo de RFM nestas condições resultaria em perda de dados e viés nos KPIs de faturamento e churn.

In [45]: *# 3.5.1. Diagnóstico de Nulos Segmentado por Dataset*

```
def report_nulos_detalhado(datasets, titles):
    """
    Gera um relatório de qualidade focado apenas em anomalias (nulos),
    separando os resultados por dataset para facilitar a limpeza cirúrgica.
    """
    for df, nome in zip(datasets, titles):
        total_rows = len(df)
        null_counts = df.isnull().sum()

        # Filtra apenas colunas que possuem ao menos 1 nulo
        null_cols = null_counts[null_counts > 0].reset_index()
        null_cols.columns = ['coluna', 'qtd_nulos']

        print(f"\n{'='*30}")
        print(f" DATASET: {nome.upper()}")
        print(f"{'='*30}")

        if not null_cols.empty:
            # Calcula percentual e ordena de forma decendente
            null_cols['%_nulos'] = (null_cols['qtd_nulos'] / total_rows) * 100
            null_cols = null_cols.sort_values(by='%_nulos', ascending=False)

            # Aplicação de estilo e exibição
            display(null_cols.style.background_gradient(cmap='YlOrRd', subset=[
                '%_nulos', 'qtd_nulos'
            ]).format({'%_nulos': '{:.2f}%', 'qtd_nulos': '{:,}'})
                    .hide(axis='index'))
        else:
            print(f"✅ Excelente! O dataset {nome} não possui valores nulos.")

    # Execução do diagnóstico para o Pós-Venda
    report_nulos_detalhado([df_vendas, df_devolucoes], ["Vendas", "Devoluções"])
```

```
=====
DATASET: VENDAS
=====
```

coluna	qtd_nulos	%_nulos
subgrupo	270,608	62.50%
centro_custo	198,293	45.80%
descricao_cc	198,293	45.80%
descricao	3,631	0.84%
grupo	3,631	0.84%
cod_produto	3,623	0.84%
filial	10	0.00%
cliente	9	0.00%
cnpj	9	0.00%
cod_cliente	9	0.00%
consultor	9	0.00%
data_emissao	9	0.00%
nota_fiscal	9	0.00%
valor_bruto	5	0.00%
valor_liquido	5	0.00%
impostos	5	0.00%
custo	5	0.00%
desconto	5	0.00%
quantidade	5	0.00%
margem_bruta_valor	5	0.00%
margem_bruta_perc	5	0.00%

=====
DATASET: DEVOLUÇÕES
=====

coluna	qtd_nulos	%_nulos
subgrupo	5,076	63.28%
centro_custo	2,451	30.56%
descricao_cc	2,451	30.56%
grupo	19	0.24%
descricao	19	0.24%
cod_produto	17	0.21%
consultor	3	0.04%
cliente	3	0.04%
cnpj	3	0.04%
cod_cliente	3	0.04%
data_emissao	3	0.04%
filial	3	0.04%
nota_fiscal	3	0.04%
valor_bruto_dev	2	0.02%
valor_liquido	2	0.02%
impostos	2	0.02%
custo	2	0.02%
desconto	2	0.02%
quantidade	2	0.02%
margem_bruta_valor	2	0.02%
margem_bruta_perc	2	0.02%

3.6. Auditoria de Duplicidade (Redundancy Check)

Nesta etapa, investigamos a existência de "Gêmeos Idênticos" nos datasets. Uma linha duplicada no Pós-Venda da INOVA pode significar um erro de processamento no ETL do Power BI ou uma falha de integração no ERP Protheus.

- **Diferenciação Técnica: * Excesso de Registros:** Indica quantas linhas a mais temos na base (ex: se um registro aparece 3 vezes, o excesso é 2).
 - **Linhas Envolvidas:** Indica o total de registros comprometidos (no exemplo acima, seriam as 3 linhas), o que nos ajuda a isolar a amostra para investigação.
- **Impacto no Negócio:** A duplicidade distorce o cálculo do **Market Share**, infla o **ROI de Peças Genuínas** e gera falsos positivos na análise de **Churn**. Se um cliente "compra" o mesmo item duas vezes no mesmo segundo via erro de sistema, nossa frequência no RFM estará enviesada.

- **Metodologia de Inspeção:** Utilizamos a ordenação por `nota_fiscal` e `cod_produto` para agrupar visualmente os registros idênticos, facilitando a confirmação de que se trata de erro de dado e não de vendas legítimas e sequenciais.

```
In [46]: ## 3.6. Qualidade de Dados: Auditoria de Duplicidade

import pandas as pd

def report_duplicados_sequencial(df, nome):
    print(f"--- 🐛 Auditoria de Duplicados: {nome} ---")

    # 1. Cálculo de Métricas Básicas
    total_linhas = len(df)
    # Identificação de TODAS as cópias (keep=False)
    df_dups = df[df.duplicated(keep=False)].copy()
    qtd_dups = df.duplicated().sum() # Apenas o "excesso"

    if not df_dups.empty:
        # 2. Ordenação Estratégica para Agrupamento Visual
        # Usamos nota_fiscal e cod_produto como âncoras de ordenação
        cols_sort = ['nota_fiscal', 'cod_produto', 'valor_liquido', 'quantidade']
        cols_sort = [c for c in cols_sort if c in df_dups.columns]

        df_dups_sorted = df_dups.sort_values(by=cols_sort)

        print(f"Total de Linhas na Base: {total_linhas}")
        print(f"Registros redundantes (Excesso): {qtd_dups} ({qtd_dups/total_linhas*100:.1f}%)")
        print(f"Total de linhas envolvidas em duplicidade: {len(df_dups)}")
        print(f"\n🔍 Amostra de 'Gêmeos Idênticos' em {nome} (sequencial):")
        display(df_dups_sorted.head(10))
    else:
        print(f"✅ {nome}: Nenhuma duplicidade exata encontrada.")
        print("\n" + "="*60 + "\n")

    # Execução para ambos os datasets
    report_duplicados_sequencial(df_vendas, "Vendas")
    report_duplicados_sequencial(df_devolucoes, "Devoluções")

--- 🐛 Auditoria de Duplicados: Vendas ---
✅ Vendas: Nenhuma duplicidade exata encontrada.

=====

--- 🐛 Auditoria de Duplicados: Devoluções ---
✅ Devoluções: Nenhuma duplicidade exata encontrada.

=====
```

3.7. Consolidação Financeira (Líquido Transacional)

Nesta etapa, realizamos a integração das bases de **Vendas** e **Devoluções** sem filtros categoriais, visando a apuração do faturamento líquido total da carteira.

- **Soberania do Saldo:** Implementamos a inversão aritmética nas devoluções ($\ast -1$). Isso garante que a função de agregação trate retornos de mercadoria como deduções reais de receita, prevenindo a inflação do faturamento.
- **Integridade de Conciliação:** O script executa um *Stress Test* comparando o montante total esperado contra o realizado após o agrupamento por CNPJ. Qualquer divergência acima de R\$ 0,0001 é reportada como erro de processamento.
- **Rastreabilidade de Órfãos:** Identificamos clientes que processaram devoluções mas não figuram na base de vendas ativa. Este insight é fundamental para separar o "Churn de Consumo" de movimentos logísticos de garantias antigas.

```
In [47]: ## 3.7. Consolidação Financeira (Líquido Transacional)

from difflib import SequenceMatcher
import pandas as pd

def similaridade(a, b):
    if pd.isna(a) or pd.isna(b): return 0
    return SequenceMatcher(None, str(a).upper(), str(b).upper()).ratio()

# 1. PARÂMETRO DE CALIBRAÇÃO
THRESHOLD_FUZZY = 0.60

# 2. Preparação e Reset
vendas_audit = df_vendas_clean.copy().reset_index(drop=True)
devolucoes_audit = df_dev_clean.copy().reset_index(drop=True)
cnpjjs_vendas_set = set(vendas_audit['cnpj'].unique())
nomes_vendas_fundo = vendas_audit['cliente'].unique()

# 3. SANEAMENTO BÁSICO (Nome Exato)
map_nome_exato = vendas_audit.groupby('cliente')['cnpj'].first().to_dict()
def saneamento_basico(row):
    if row['cnpj'] not in cnpjjs_vendas_set and row['cliente'] in map_nome_exato:
        return map_nome_exato[row['cliente']]
    return row['cnpj']

devolucoes_audit['cnpj'] = devolucoes_audit.apply(saneamento_basico, axis=1)
orfaos_atuais = devolucoes_audit[~devolucoes_audit['cnpj'].isin(cnpjjs_vendas_set)]

# 4. PROCESSAMENTO FUZZY E ATRIBUIÇÃO
registros_fuzzy = []
for _, row in orfaos_atuais.iterrows():
    nome_o, cnpj_o = row['cliente'], row['cnpj']
    melhor_score, candidato = 0, "Não Encontrado"
    for nome_v in nomes_vendas_fundo:
        score = similaridade(nome_o, nome_v)
        if score > melhor_score:
            melhor_score, candidato = score, nome_v

    cnpj_novo, status_atribuicao = None, "Pendente"
    if melhor_score >= THRESHOLD_FUZZY:
        cnpj_novo = vendas_audit[vendas_audit['cliente'] == candidato]['cnpj'].i
        status_atribuicao = "Atribuído"

    registros_fuzzy.append({
        'Nome Orfao': nome_o, 'Candidato em Vendas': candidato, 'Score': melhor_
        'Status': status_atribuicao, 'CNPJ Original': cnpj_o, 'CNPJ Atribuido':
    })
```

```

df_balanco_clientes = pd.DataFrame(registros_fuzzy)

# 5. APLICAÇÃO DO SANEAMENTO
map_final = df_balanco_clientes[df_balanco_clientes['Status'] == 'Atribuído'].se
devolucoes_audit['cnpj'] = devolucoes_audit['cnpj'].replace(map_final)

# 6. UNIFICAÇÃO DE ESTRUTURA E SINAIS
if 'valor_bruto_dev' in devolucoes_audit.columns:
    devolucoes_audit = devolucoes_audit.rename(columns={'valor_bruto_dev': 'valo

# Lista de colunas que devem ser invertidas (Deduções)
cols_financeiras = ['valor_bruto', 'valor_liquido', 'quantidade', 'custo', 'impo

for col in cols_financeiras:
    if col in devolucoes_audit.columns:
        devolucoes_audit[col] = devolucoes_audit[col] * -1

# Criamos o Master com todas as colunas (alinhamento automático por nome)
df_transacional_master = pd.concat([vendas_audit, devolucoes_audit], ignore_inde

# 7. OUTPUT DE AUDITORIA
venda_bruta_total = vendas_audit['valor_liquido'].sum()
devolucao_total = devolucoes_audit['valor_liquido'].sum() # Já está negativo
faturamento_esperado = venda_bruta_total + devolucao_total

print(f"--- 📋 Auditoria de Unificação Financeira (Master 21 Colunas) ---")
print(f"{'Vendas Totais (Gross)':<35} | R$ {venda_bruta_total:,.2f}")
print(f"{'Devoluções Totais (Returns)':<35} | R$ {abs(devolucao_total):,.2f}")
print(f"{'Faturamento Líquido Realizado':<35} | R$ {df_transacional_master['valo
print(f"{'Diferença de Conciliação':<35} | R$ {abs(faturamento_esperado - df_tra
print(f"{'Total de Linhas no Master':<35} | {len(df_transacional_master):,}")
print(f"{'Total de Colunas Final':<35} | {len(df_transacional_master.columns)}")
print("-" * 65)

if not df_balanco_clientes.empty:
    print(f"\n🔍 Tabela de Calibragem Fuzzy (Atribuídos: {len(df_balanco_cliente
    display(df_balanco_clientes.sort_values(by='Score', ascending=False)
        .style.map(lambda val: 'background-color: #2e7d32; color: white; fon
        .format({'Score': '{:.2%}'))

```

--- 📋 Auditoria de Unificação Financeira (Master 21 Colunas) ---

Vendas Totais (Gross)	R\$ 653,301,133.83
Devoluções Totais (Returns)	R\$ 30,231,998.55
Faturamento Líquido Realizado	R\$ 623,069,135.28
Diferença de Conciliação	R\$ 0.000000
Total de Linhas no Master	440,967
Total de Colunas Final	21

🔍 Tabela de Calibragem Fuzzy (Atribuídos: 15)

	Nome Orfao	Candidato em Vendas	Score	Status	CNPJ Original	CNPJ Al
0	ARCELORMITTAL BIO FLORESTAS LTDA	ARCELORMITTAL BIOFLORESTAS LTDA	98.41%	Atribuído	13163645002998	13163645
3	VITAL ENGENHARIA AMBIENTAL SA	VITAL ENGENHARIA AMBIENTAL S/A	98.31%	Atribuído	2536066000800	25360660
2	ABR SERVICOS FLORESTAIS LTDA	ABR SERVIÇOS FLORESTAIS LTDA	96.43%	Atribuído	5215135000279	52151350
4	PORTO SUDESTE DO BRASIL S/A	PORTO SUDESTE DO BRASIL S A	96.30%	Atribuído	8310839000138	83108390
6	ZOCAR RIO CAMINHOES LTDA	ZOCAR RIO CAMINHÕES LTDA	95.83%	Atribuído	3096738000869	30967380
13	OECI S.A	OECI SA	93.33%	Atribuído	10220039007504	10220039
8	CITY CAR VEICULOS SERVICOS E MINER LTDA	CITY CAR VEICULOS SERVICOS E MINERACAO	90.91%	Atribuído	65287872000390	65287872
11	SANTO ALEIXO MG EMPREENDIMENTOS AGROPECUÁRIOS LTDA	SANTO ALEIXO EMPREENDIMENTOS AGROPECUARI	86.67%	Atribuído	73198574000351	73198574
12	TRATORSUL EMPREENDIMENTOS E LOCACAO DE MAQUINAS LTDA	TRATORSUL EMPREENDIMENTOS E LOCACAO DE	84.78%	Atribuído	12754975000275	12754975
1	U&M MINERACAO E CONSTRUCAO S/A	U&M MINERAÇÃO E CONSTRUÇÃO S.A	83.33%	Atribuído	18540906003260	18540906
5	ITAETE MOVIMENTACAO LTDA	ITAETE MOVIMENTACAO LOGISTICA LTDA	81.36%	Atribuído	5685282000806	56852820
14	LUIZ ANTONIO MALTA	LUIZ ANTONIO MAXIMIANO	80.00%	Atribuído	8598391689	39439168
9	SKAVAMINAS MIN,CONST. E TRANSP LTDA	SKAVA MINAS MINERACAO, CONSTRUCOES E TRANSPORTES LTDA	77.27%	Atribuído	3353341000724	33533410
7	W.P.X. LOCACOES S.A.	W.P.X LOCAÇÕES LTDA	71.79%	Atribuído	22212519000842	22212519
10	ARTUR AUGUSTO DE LIMA NESSRALLA	ARTUR OTAVIO DE OLIVEIRA NETO	60.00%	Atribuído	97908088600	78108088

3.8. Consolidação do Golden Record (Identidade Global por Frequência)

Nesta etapa, para resolver a fragmentação cadastral implementamos um **Motor de Identidade Global**.

Lógica de Soberania Absoluta:

- **DNA Nominal:** O algoritmo extrai o radical das empresas através de uma normalização agressiva, ignorando ruídos como "LTDA", "S/A", numerais, conectores e pontuações gráficas.
- **Critério de Frequência:** Para cada cluster de nomes similares (Threshold 96%), o sistema elege automaticamente como **Mestre do Grupo** o CNPJ que possui o maior volume histórico de transações, garantindo que a matriz ou a unidade mais relevante dê o nome ao grupo.
- **Unificação Transacional:** Todas as vendas e devoluções são enriquecidas com as colunas `grupo_economico` e `cnpj_grupo_economico`, permitindo uma visão consolidada do faturamento real por cliente, independentemente de quantas filiais ele possua.

Vantagens para o Negócio:

1. **Fim da Fragmentação:** Clientes que comprem por diferentes CNPJs são vistos como uma única conta estratégica.
2. **Precisão de Atendimento:** Facilita a gestão dos Consultores, que atendem o grupo econômico como um todo.
3. **Dados Prontos para Campanha:** Elimina a necessidade de saneamentos manuais para análises de Churn e ROI.

Resultado: Esta abordagem entrega o "Dataset de Ouro", onde a integridade fiscal (CNPJ original) convive harmonicamente com a inteligência de negócio (Grupo Econômico).

```
In [48]: # --- 3.8. Motor de Identidade Global (Versão Consolidada e Otimizada) ---
import unicodedata
import re
from difflib import SequenceMatcher
import pandas as pd

def normalizar_dna_puro(texto):
    if pd.isna(texto): return ""
    texto = unicodedata.normalize('NFKD', str(texto)).encode('ASCII', 'ignore').decode()
    # Limpeza de DNA: remove sufixos, pontuação e conectores
    texto = re.sub(r'\b(LTDA|LTD|S/?A|EIRELI|ME|EPP|CIA|COMPANHIA|FILIAL|MATRIZ|', '')
    texto = re.sub(r'COES\b', 'CAO', texto); texto = re.sub(r'OES\b', 'AO', texto)
    texto = re.sub(r'([S])S\b', r'\1', texto)
    return re.sub(r'^A-Z', '', texto)

# 1. Ranking de Frequência (Vetorizado)
# Usamos o faturamento histórico para decidir quem manda no grupo
freq = df_transacional_master['cnpj'].value_counts().to_dict()

# 2. Preparação de Identidades
# Mudamos de 'cliente_consolidado' para 'cliente' que é a coluna original do seu
df_identities = df_transacional_master[['cnpj', 'cliente']].drop_duplicates().c
```

```

df_identidades['frequencia'] = df_identidades['cnpj'].map(freq)
df_identidades['dna'] = df_identidades['cliente'].apply(normalizar_dna_puro)
df_identidades['letra'] = df_identidades['dna'].str[0]

# Ordena pelo mestre mais frequente para garantir a soberania
df_identidades = df_identidades.sort_values(['letra', 'frequencia'], ascending=[

THRESHOLD = 0.96
map_final_nome = {}
map_final_cnpj = {}
auditoria = []

print(f" ⚡ Processando {len(df_identidades)} identidades via Bloqueio Alfabético

# 3. Processamento por Blocos
for letra, frame in df_identidades.groupby('letra'):
    if not letra: continue

    dnas = frame['dna'].tolist()
    cnpjs = frame['cnpj'].tolist()
    nomes = frame['cliente'].tolist()

    visitados_bloco = set()

    for i in range(len(dnas)):
        if cnpjs[i] in visitados_bloco: continue

        for j in range(i + 1, len(dnas)):
            if cnpjs[j] in visitados_bloco: continue

            # Comparação de DNA Nominal
            score = 1.0 if dnas[i] == dnas[j] else SequenceMatcher(None, dnas[i]

            if score >= THRESHOLD:
                map_final_nome[cnpjs[j]] = nomes[i]
                map_final_cnpj[cnpjs[j]] = cnpjs[i]
                visitados_bloco.add(cnpjs[j])
                auditoria.append({'Filial': nomes[j], 'Mestre': nomes[i], 'Score

# 4. Atribuição Final ao Master
# Criamos as novas colunas baseadas na coluna 'cliente'
df_transacional_master['grupo_economico'] = df_transacional_master['cnpj'].map(m
df_transacional_master['cnpj_grupo_economico'] = df_transacional_master['cnpj'].

print(f" ✔ Concluído! Grupos únicos identificados: {df_transacional_master['cnpj
if auditoria:
    print(f" ✔ Foram realizadas {len(auditoria)} unificações automáticas.")
    display(pd.DataFrame(auditoria))

```

- ⚡ Processando 4006 identidades via Bloqueio Alfabético...
- ✔ Concluído! Grupos únicos identificados: 3955
- ✔ Foram realizadas 40 unificações automáticas.

	Filial	Mestre	Score
0	ARCELORMITTAL BIOFLORESTAS LTDA	ARCELORMITTAL BIOFLORESTAS LTDA	1.000000
1	ACTECH ALUMINA CHEMICAL TECHNOLOGY LTDA.	ACTECH ALUMINA CHEMICAL TECHNOLOGY LTDA	1.000000
2	ABR SERVICOS FLORESTAIS LTDA	ABR SERVIÇOS FLORESTAIS LTDA	1.000000
3	ALBERIONE JOSE CAMPOS	ALBERIONE JOSE CAMPOS	1.000000
4	BRAM OFFSHORE TRANSPORTES MARITIMO	BRAM OFFSHORE TRANSPORTES MARITIMOS LTDA	1.000000
5	CSN MINERACAO S.A.	CSN MINERACAO S.A.	1.000000
6	COPAVI CONSERVAÇÃO E PAVIMENTAÇÃO DE RODOVIAS...	COPAVI CONSERVACAO E PAVIMENTACAO DE RODOVIAS ...	1.000000
7	CONSORCIO ETHOS/HWN RODOVIA MG188	CONSORCIO ETHOS/HWN RODOVIA MG428	1.000000
8	COOPERATIVA DOS PRODUTORES RURAIS DE POMPEU LTDA	COOPERATIVA DOS PRODUTORES RURAIS DE POMPEU LTDA	1.000000
9	DIRCEU JULIO GATTO	DIRCEU JÚLIO GATTO	1.000000
10	ELLEN DE FATIMA AMADI BARROS	ELLEN DE FATIMA AMADI BARROS	1.000000
11	ERALDO DE OLIVEIRA	ERALDO DE OLIVEIRA LTDA	1.000000
12	EDUARDO JUNIO SOUZA SANTOS	EDUARDO JUNIO SOUZA SANTOS	1.000000
13	HEDENILSON ELIAS DE SOUZA 11725024632	HEDENILSON ELIAS DE SOUZA	1.000000
14	ITERCELER LOGISTICA LTDA	ITERCELER LOGISTICA S/A	1.000000
15	IPE ENGENHARIA E CONSTRUCOES LTDA	IPE ENGENHARIA E CONSTRUCOES LTDA	1.000000
16	IDEAL ENGENHARIA LTDA	IDEAL ENGENHARIA LTDA	1.000000
17	IVAN JOSE DOS SANTOS	IVAN JOSE DOS SANTOS	1.000000
18	LEANDRO GONÇALVES SILVA	LEANDRO GONCALVES DA SILVA	1.000000
19	LUIZ ALBERTO PEREIRA	LUIZ ALBERTO PEREIRA	1.000000
20	LUIZ ANTONIO MAGELA CONRADO	LUIZ ANTONIO MAGELA CONRADO	1.000000
21	LUCINEI MARIA DA SILVA	LUCINEI MARIA DA SILVA 07504017612	1.000000
22	MATEUS DALFIOR	MATEUS DALFIOR	1.000000
23	MICHEL RODRIGUES BLANCO	MICHEL RODRIGUES BLANCO LTDA	1.000000
24	MARCOS MACIEL SOARES	MARCOS MACIEL SOARES	1.000000
25	NILDA ALVES DA SILVA	NILDA ALVES DA SILVA	1.000000
26	PORTO SUDESTE DO BRASIL S/A	PORTO SUDESTE DO BRASIL S A	0.972973
27	RIMA INDUSTRIAL S/A	RIMA INDUSTRIAL SA	1.000000

	Filial	Mestre	Score
28	RD TRANSPORTE E LOCACAO LTDA	RD TRANSPORTES E LOCACOES LTDA	1.000000
29	55.523.795 ROBERTO APARECIDO FERREIRA DA SILVA	ROBERTO APARECIDO FERREIRA DA SILVA	1.000000
30	SANTO ALEIXO MG EMPREENDEIMENTOS AGROPECUÁRIOS ...	SANTO ALEIXO EMPREENDEIMENTOS AGROPECUARI	0.960000
31	TERRABEL EMPREENDEIMENTOS LTDA	TERRABEL EMPREENDEIMENTOS LTDA	1.000000
32	TAMASA ENGENHARIA S.A.	TAMASA ENGENHARIA S/A	0.969697
33	U&M MINERACAO E CONSTRUCAO S/A	U&M MINERAÇÃO E CONSTRUÇÃO S.A	0.976744
34	UEDER BATISTA DOS SANTOS	UEDER BATISTA DOS SANTOS	1.000000
35	VITAL ENGENHARIA AMBIENTAL SA	VITAL ENGENHARIA AMBIENTAL S/A	1.000000
36	VITAL ENGENHARIA AMBIENTAL S.A.	VITAL ENGENHARIA AMBIENTAL S/A	0.979592
37	VIX LOGISTICA S/A	VIX LOGISTICA S/A	1.000000
38	WANDERLEY BRANTES FERREIRA JUNIOR	WANDERLEY BRANTES FERREIRA JUNIOR	1.000000
39	ZOCAR RIO CAMINHÕES LTDA	ZOCAR RIO CAMINHÕES LTDA	1.000000

3.9. Exportação em Formato de Alta Performance (.parquet)

Para a entrega final do **Golden Dataset**, optamos pelo formato **Apache Parquet**. Esta escolha estratégica visa otimizar as etapas subsequentes de visualização e análise de dados.

Benefícios do Formato Colunar:

- **Integração com Power BI:** O carregamento de dados é acelerado pela estrutura colunar, reduzindo o tempo de atualização dos Dashboards.
- **Tipagem Estrita:** Diferente do CSV, o Parquet armazena os metadados dos tipos de colunas, garantindo que datas e valores financeiros sejam interpretados corretamente sem intervenção manual.
- **Eficiência de Armazenamento:** Alta taxa de compressão sem perda de informação, facilitando o tráfego de dados em rede.

```
In [49]: # --- 3.9. Exportação para Formato de Alta Performance ---

# Definindo o caminho e nome do arquivo
nome_arquivo_parquet = "INOVA_Master_Vendas_Consolidado.parquet"

try:
    # Exportação com compressão snappy (equilíbrio ideal entre velocidade e tama
    df_transacional_master.to_parquet(nome_arquivo_parquet, compression='snappy')

import os
tamanho_mb = os.path.getsize(nome_arquivo_parquet) / (1024 * 1024)
```

```

print(f"✅ Arquivo exportado com sucesso: {nome_arquivo_parquet}")
print(f"📦 Tamanho final do arquivo: {tamanho_mb:.2f} MB")
print(f"🚀 O dataset está pronto para ser consumido pelo Power BI ou outros

except Exception as e:
    print(f"❌ Erro na exportação: {e}")
    print("Dica: Certifique-se de ter as bibliotecas 'pyarrow' ou 'fastparquet'

```

✅ Arquivo exportado com sucesso: INOVA_Master_Vendas_Consolidado.parquet
 📦 Tamanho final do arquivo: 15.40 MB
 🚀 O dataset está pronto para ser consumido pelo Power BI ou outros Data Products.

3.10. Conclusão

```

In [50]: # --- 3.10. Conclusão do ETL ---

print(f"--- 📊 Relatório de Finalização do Pipeline (etl_vendas_devolucoes) ---")
print(f"📅 Data do Processamento: {pd.Timestamp.now().strftime('%d/%m/%Y %H:%M:%S')}")
print(f"📄 Arquivo Gerado: {nome_arquivo_parquet}")
print(f"📊 Total de Registros Processados: {len(df_transacional_master):,}")
print(f"🏢 Total de Grupos Econômicos: {df_transacional_master['cnpj_grupo_economico'].nunique():,}")
print(f"💰 Faturamento Líquido Consolidado: R${df_transacional_master['valor_liquido'].sum():,}")
print("-" * 65)
print("ETL executado de ponta a ponta com sucesso.")

--- 📊 Relatório de Finalização do Pipeline (etl_vendas_devolucoes) ---
📅 Data do Processamento: 13/03/2026 11:48:34
📄 Arquivo Gerado: INOVA_Master_Vendas_Consolidado.parquet
📊 Total de Registros Processados: 440,967
🏢 Total de Grupos Econômicos: 3,955
💰 Faturamento Líquido Consolidado: R$ 623,069,135.28
-----
ETL executado de ponta a ponta com sucesso.

```